

Mission Control Dashboard

Agent-Orchestrierung & Echtzeit-Visualisierung

Konzept für die nächste Generation des OpenClaw Dashboards

Bastian Steppa | 25. April 2026

1. Executive Summary

Das Mission Control Dashboard ist die zentrale Kommandozentrale für die gesamte OpenClaw-Agent-Infrastruktur. Während das aktuelle Agent Dashboard (dashboard.steppa.online) primär eine Übersicht über vergangene und laufende Agent-Session bietet, geht das Mission Control einen entscheidenden Schritt weiter: Es visualisiert den **lebendigen, pulsierenden Datenstrom** einer KI-Orchestrierung in Echtzeit.

Das Ziel ist nicht nur Monitoring – es ist das **Erlebnis** einer Architektur beim Arbeiten zuzusehen. Jeder Subagent-Spawn, jeder Token-Fluss, jeder Status-Übergang wird sichtbar, fühlbar, greifbar. Das Dashboard wird zur Nabelschnur zwischen Mensch und Maschine – ein Fenster in die Gedanken eines verteilten KI-Systems.

Zielgruppe: Bastian (Operator) und andere Nutzer der OpenClaw-Plattform, die Einblick in die Arbeitsweise ihrer KI-Orchestrierung wollen.

Vision: Eine Oberfläche, bei der man nicht nur sieht, was passiert – sondern spürt, wie das System denkt.

2. Aktuelle Architektur

Die OpenClaw-Infrastruktur basiert auf einer dreistufigen Agent-Orchestrierung:

2.1 Die drei Ebenen

Ebene 1: Der Proxy (Main Agent)

Der Telegram-Bot @ogLobster_bot ("Bernd") empfängt Benutzeranfragen. Der Main Agent agiert als reines Frontend – er bestätigt Eingänge, delegiert und liefert Ergebnisse aus. Er schreibt keinen Code, führt keine Recherche durch, löst keine Probleme. Er ist der glorifizierte, zynische Briefkasten zwischen Mensch und Maschine.

Ebene 2: Der Orchestrator

Bei komplexen Aufgaben (Multi-File, neue Scripts, System-Architektur) spawnt der Main Agent einen Orchestrator-Subagent. Dieser analysiert die Task, wählt aus der Modell-Matrix das beste Modell für den Job und delegiert an spezialisierte Worker-Subagents. Der Orchestrator ist Middle-Management – er bricht Arbeit herunter, überwacht und aggregiert Ergebnisse.

Ebene 3: Die Subagents (Worker)

Isolierte Worker-Sessions, die jeweils eine spezifische Teilaufgabe bearbeiten. Jeder Subagent erhält ein für seine Aufgabe optimiertes Modell aus dem Alibaba Coding Plan Pool. Nach Abschluss (oder Timeout nach 15 Minuten) wird das Ergebnis an den Orchestrator zurückgemeldet.

2.2 Infrastruktur

Komponente	Technologie
Server	Hetzner VPS vmd190638
OS	Linux 6.8.0-110-generic (x64)
Node.js	v22.22.1
OpenClaw	v2026.4.15
Gateway	Port 18789 (loopback)
API-Server	Port 3002 (Dashboard API)
Telegram Bot	@ogLobster_bot (Bernd)

2.3 Live-Domains

- `dashboard.steppa.online` – Agent Dashboard (bestehend)
 - `apps.steppa.online` – Apps Index (16 Apps)
 - `config.steppa.online` – Config Editor
-

3. Verfügbare Datenquellen

3.1 Agent-Daten (Dashboard API)

Jeder Agent-Spawn und jede Completion erzeugt strukturierte Daten:

Beim Start (`POST /api/agents/start`):

- `sessionKey` – Eindeutige Session-ID
- `label` – Bezeichnung des Agents (z.B. "Orchestrator", "Frontend Coder")
- `task` – Kurze Beschreibung der Aufgabe
- `model` – Zugewiesenes Modell (z.B. `bailian/qwen3.6-plus`)
- `timestamp` – Startzeitpunkt

Beim Ende (`POST /api/agents/end`):

- `status` – done, failed, timeout
- `runtimeMs` – Laufzeit in Millisekunden
- `tokens` – Verarbeitete Token (input/output/total)

Listen-Endpunkt (`/api/agents/list`):

- Alle aktiven und jüngsten Agents
- Parent-Child-Beziehungen
- Pending Descendants
- Kumulierte Token-Statistiken

3.2 System-Daten

Datenpunkt	Quelle	Intervall
Gateway Health	<code>curl http://127.0.0.1:18789/health</code>	30 Sekunden
RAM/Swap	<code>free -m</code>	30 Sekunden
CPU-Last	<code>/proc/stat</code>	30 Sekunden
Festplatte	<code>df -h</code>	5 Minuten
Uptime	<code>uptime</code>	5 Minuten

3.3 Watchdog-Daten

Der Telegram-Watchdog (alle 15 Minuten) protokolliert:

- Gateway Health-Status
- Telegram API Reachability
- sendMessage-Test-Ergebnis
- PID des aktiven Gateway-Prozesses

3.4 ElevenLabs-API

- Character-Usage: Verbrauchte vs. verfügbare Zeichen (39.589/Monat)
- Voice-Library: 4 Custom Voices, 21 Premade
- Instant Voice Cloning: Enabled

3.5 Caddy & Deployments

- Aktive Domains und Reverse-Proxy-Routen
 - SSL-Zertifikatsstatus
 - Upstream-Server-Status
-

4. Visualisierungskonzept

4.1 Gesamt-Layout

Das Mission Control Dashboard ist als Fullscreen-Application konzipiert – optimiert für große Monitore, mit sanftem Responsive-Fallback für mobile Geräte. Das Layout besteht aus fünf Zonen:

Zone 1: Top-Bar (Global Status)

Die obere Leiste ist das ständige Nervenzentrum. Sie zeigt auf einen Blick:

- **Live-Indikator:** Pulsierender grüner Punkt mit "LIVE"-Label. Bei Gateway-Fail wechselt er zu rot mit letztem bekannten Status-Zeitstempel.
- **Agent-Counter:** Aktive Agents | Completed Today | Failed Today | Avg Runtime
- **System-Health:** Mini-Indikatoren für RAM, CPU, Gateway, Telegram
- **Uhrzeit & Datum:** Live-Uhr (Europa/Berlin)
- **Token-Tagesverbrauch:** Gesamte Tokens aller Agents heute

Zone 2: Pipeline-Ansicht (Hauptbereich, ca. 60% der Breite)

Das Herzstück. Eine horizontale, links-nach-rechts fließende Pipeline, die den aktuellen Arbeitsfluss visualisiert:

- **Links:** Incoming Queue – User-Nachrichten die eingehen und auf Verarbeitung warten
- **Mitte-Links:** Orchestrator-Nodes – Aktuell aktive Orchestratoren, pulsierend mit Runtime-Timer
- **Mitte:** Verbindungslinien – Animierte Datenflüsse zwischen Orchestrator und Workern. Jede Linie repräsentiert eine Task-Delegation. Datenpartikel fließen entlang der Linie – dicker = mehr Token-Transfer.
- **Mitte-Rechts:** Worker-Nodes – Die Subagents, die aktuell arbeiten. Jeder Node ist eine Karte mit Label, Modell-Farbe, Runtime-Timer und Status.
- **Rechts:** Output-Queue – Fertige Ergebnisse die auf Auslieferung warten

Zone 3: Worker-Grid (rechte Seitenleiste, ca. 25%)

Vertikales Grid aller aktuell laufenden Workers:

- Karte pro Worker: Label, Modell (farbkodiert), Runtime, Token-Counter, Progress-Balken
- Hover: Tool-Calls, aktuelle Activity, Parent-Orchestrator
- Klick: Details im Modal

Zone 4: Live-Event-Stream (rechte Seitenleiste unten, ca. 15%)

Scrollender Feed aller Ereignisse in Echtzeit:

- [03:17:42] ● "Frontend Dev" completed (12s, 43k tokens)
- [03:17:41] ● "Researcher" spawned (glm-5)
- [03:17:40] ● "Backend Dev" running (3m 21s, 127k tokens)
- [03:17:35] ● "QA Engineer" failed (timeout nach 15m)

Zone 5: Analytics-Overlay (optional, full-screen)

Überlagerung mit historischen Analysen – aktivierbar per Toggle oder Hotkey.

4.2 Farbsystem

Jedes Modell erhält eine eindeutige Farbe für sofortige visuelle Zuordnung:

Modell	Farbe	Hex	Assoziation
bailian/qwen3.6-plus	Gold	#FFD700	Primary, Premium
bailian/qwen3.5-plus	Cyan	#00E5FF	Fallback, Cool
bailian/qwen3-coder-plus	Orange	#FF6D00	Coding, Warm
bailian/qwen3-coder-next	Rot-Orange	#FF3D00	Cutting Edge
bailian/glm-5	Blau	#2979FF	Tool-Calling, Trust
bailian/kimi-k2.5	Violett	#AA00FF	Vision, Kreativ
bailian/MiniMax-M2.5	Grün	#00E676	Bulk, Effizienz

Status-Farben:

- ● Running: #00E676 (Neon-Grün mit Glow)
- ● Done: #2979FF (Blau, gedeckt)
- ● Spawning: #FFD740 (Gold, pulsierend)
- ● Failed: #FF1744 (Rot, Alarm)
- ● Timeout: #FF9100 (Orange, Warning)

UI-Hintergrund:

- Primary: #0A0A0F (Near-Black, tiefes Raum-Schwarz)
- Secondary: #12121A (Karten-Hintergrund)
- Border: #1E1E2E (subtile Trennung)

- Text Primary: #E0E0E8
 - Text Secondary: #8888AA
 - Accent Glow: rgba(255, 215, 0, 0.15)
-

5. Pipeline-Visualisierung im Detail

5.1 Der Lebenszyklus eines Tasks

Jede User-Anfrage durchläuft einen sichtbaren Weg durch das System. Der Benutzer soll diesen Weg nicht nur verstehen – er soll ihn **sehen**:

Phase 1: Eingang (Incoming)

Die User-Nachricht erscheint als neue Karte in der Incoming-Queue. Sie enthält:

- Zeitstempel des Eingangs
- Kanal (Telegram, Webchat, Discord)
- User-Label (Bastian, @Steppa_tg)
- Kurze Task-Zusammenfassung (erste 80 Zeichen)

Die Karte pulsiert leicht (Waiting-Animation) bis der Main Agent reagiert.

Phase 2: Proxy-Processing

Der Main Agent bestätigt den Eingang. Visuell: Die Karte bewegt sich nach rechts und verschmilzt mit dem Orchestrator-Node. Eine kurze "Ack"-Animation (grüner Blitz) signalisiert Bestätigung.

Phase 3: Orchestrierung

Der Orchestrator-Node erscheint in Gold (#FFD700). Er pulsiert mit höherer Frequenz während er analysiert. Der Runtime-Timer startet. Über dem Node steht das Label "Orchestrator".

Phase 4: Worker-Spawn

Der Orchestrator teilt die Aufgabe. Neue Worker-Nodes erscheinen wie aus dem Nichts – sie "materialisieren" sich mit einer Fade-In + Scale-Up Animation. Jeder Worker erhält:

- Seine Modell-Farbe
- Ein Label ("Frontend Dev", "QA Engineer", etc.)
- Einen Runtime-Timer (beginnt bei 0:00)
- Einen Token-Counter (beginnt bei 0)

Verbindungslinien zeichnen sich vom Orchestrator zu jedem Worker – animiert, als würden sie "wachsen".

Phase 5: Parallel-Processing

Die Worker arbeiten parallel. Visuelle Indikatoren:

- **Token-Stream:** Partikel fließen entlang der Verbindungslinien. Dicker Strom = viele Tokens. Ruhiger Fluss = normaler Betrieb.
- **Activity-Puls:** Jeder Worker pulsiert leicht während er arbeitet. Stärkerer Puls = intensive Tool-Calls (exec, read, edit).
- **Progress-Bar:** Unter jedem Worker eine dezente Fortschrittsanzeige (wenn einschätzbar).
- **Runtime-Counter:** Läuft live hoch. Nähert sich 15 Minuten → Farbe wechselt zu Orange (Warning).

Phase 6: Completion

Ein Worker finished. Die Completion-Animation:

- Node-Border leuchtet grün auf
- "✅ Done" erscheint kurz
- Node wird transparenter, verschiebt sich in eine "Completed"-Zone
- Ergebniszusammenfassung erscheint im Live-Event-Stream

Phase 7: Aggregation & Delivery

Der Orchestrator sammelt alle Worker-Ergebnisse. Wenn alle fertig:

- Orchestrator-Node pulsiert einmal kräftig
- "Results Aggregated" Animation
- Verbindung zum Output-Bereich
- Ergebnis wird an den User ausgeliefert (Telegram-Icon erscheint)

Phase 8: Cleanup

Nodes verschwinden mit Fade-Out. Session wird archiviert. Statistik wird aktualisiert.

5.2 Timeout-Szenario

Wenn ein Worker die 15-Minuten-Grenze erreicht:

1. **14:00 Min:** Node-Border wird Orange (Warning)
 2. **14:30 Min:** Node beginnt zu "flackern" (Instabilitäts-Indikator)
 3. **15:00 Min:** Node wird Rot, "🕒 Timeout" erscheint
 4. **Cleanup:** Node wird auf "timeout" gesetzt, Event-Stream meldet es
 5. **Auto-Restart (optional):** Wenn konfiguriert, erscheint ein neuer Worker-Node mit dem Label "[Retry]"
-

6. Live-Event-Stream

Der Event-Stream ist das EKG des Systems. Er zeigt jeden Herzschlag in Echtzeit.

6.1 Event-Typen

Event	Icon	Farbe	Beispiel
Agent Spawned		Blau	"Frontend Dev" spawned (qwen3.6-plus)
Agent Completed		Grün	"QA Engineer" completed (12s, 43k tokens)
Agent Failed		Rot	"Backend Dev" failed (error: connection timeout)
Agent Timeout		Orange	"Researcher" timeout (15m exceeded)
Task Delegated		Cyan	Orchestrator → 3 Workers delegated
Results Aggregated		Gold	3/3 results aggregated
User Input		Weiß	Bastian: "Baue React App mit Tests"
System Alert		Gelb	Gateway restart detected
Watchdog OK		Grün	Watchdog check passed (all 3/3)

6.2 Design

- **Monospace-Font** für Zeitstempel (03:17:42)
- **Icons** vor jedem Event
- **Farbcodiert** nach Event-Typ
- **Auto-Scroll** – neues Event unten, alte scrollen nach oben
- **Pause-Button** – Stream anhalten zum Lesen
- **Filter** – Nach Typ filtern (nur Errors, nur Completions, etc.)

- **Suche** – Nach Agent-Label suchen
-

7. Analytics & Historische Analysen

7.1 Top-Bar Analytics (immer sichtbar)

- **Agents Today:** Anzahl aller gestarteten Agents
- **Completed / Failed Ratio:** Erfolgsquote
- **Average Runtime:** Durchschnittliche Laufzeit
- **Total Tokens:** Tagesverbrauch aller Tokens
- **Peak Parallelism:** Maximale gleichzeitige Agents heute

7.2 Analytics-Overlay (optional, aktivierbar)

Diagramm 1: Parallelitäts-Level (Balkendiagramm über Zeit)

- X-Achse: Zeit (Stunden des Tages)
- Y-Achse: Anzahl aktiver Agents
- Zeigt wann das System unter Last war
- Peak-Erkennung mit Markierung

Diagramm 2: Runtime-Distribution (Histogramm)

- X-Achse: Runtime (Sekunden, logarithmisch)
- Y-Achse: Anzahl Agents
- Zeigt ob Agents schnell oder langsam arbeiten
- Outlier-Markierung (über 10 Minuten)

Diagramm 3: Modell-Performance-Vergleich (Bar-Chart)

- X-Achse: Modelle (qwen3.6-plus, glm-5, etc.)
- Y-Achse: Durchschnittliche Runtime + Erfolgsquote
- Zeigt welches Modell für welche Aufgaben am effizientesten ist

Diagramm 4: Error Rate über Zeit (Line-Chart)

- X-Achse: Zeit (Tage)
- Y-Achse: Fehlerquote (%)
- Mit Moving Average (7-Tage-Glättung)
- Threshold-Linie (ab 5% = Alert)

Diagramm 5: Session-History (Gantt-Chart)

- X-Achse: Zeit (24 Stunden)
- Y-Achse: Agent-Sessions als Balken
- Jeder Balken = eine Session (Start bis Ende)
- Farbkodiert nach Modell
- Überlappende Balken = parallele Ausführung
- Visualisiert den "Rhythmus" des Systems über den Tag

Diagramm 6: Token-Verbrauch über Zeit (Area-Chart)

- X-Achse: Zeit (Tage)
- Y-Achse: Tokens pro Tag
- Gestapelt nach Input/Output
- Mit ElevenLabs-Quota-Overlay (wenn relevant)

7.3 Zusammenfassungen

- **Tages-Zusammenfassung:** Um 0:00 Uhr automatisch generiert
 - **Wochen-Report:** Montags, per Telegram zuschickbar
 - **Insights:** Automatisch erkannte Muster ("Durchschnittliche Runtime steigt", "glm-5 hat heute beste Quote")
-

8. UI/UX Design-Philosophie

8.1 Design-Sprache

"Control Room Aesthetic" – Die Ästhetik eines NASA-Kontrollraums trifft auf modernes SaaS-Dashboard.

- **Tiefes Schwarz** als primärer Hintergrund – keine Ablenkung, volle Immersion
- **Neon-Glows** als Status-Indikatoren – subtil, aber spürbar
- **Glassmorphism** für Karten und Overlays – Transparenz schafft Tiefe
- **Subtile Animationen** – nichts ist statisch, alles lebt leicht
- **Kein visueller Lärm** – jedes Element hat einen Zweck, keine Dekoration ohne Information

8.2 Typografie

- **UI-Text:** Inter oder System-UI – klar, modern, gut lesbar
- **Zahlen & Daten:** JetBrains Mono oder Fira Code – Monospace für Alignment
- **Überschriften:** Inter, bold – klare Hierarchie
- **Event-Stream:** JetBrains Mono – Terminal-Feeling

8.3 Animationen (Framer Motion)

Animation	Wann	Dauer	Easing
Spawn-Materialize	Neuer Agent erscheint	400ms	spring (stiffness: 200)
Completion-Pulse	Agent finished	600ms	easeOut
Fade-Out	Agent verschwindet	300ms	easeIn
Line-Grow	Verbindungsline zeichnet sich	500ms	easeInOut
Particle-Flow	Token-Daten fließen	kontinuierlich	linear
Heartbeat-Puls	Live-Indikator	2s loop	easeInOut
Warning-Flackern	Timeout nah	500ms loop	steps(3)
Slide-In	Event-Stream neuer Eintrag	200ms	easeOut
Scale-Up	Modal öffnet sich	300ms	spring

8.4 Responsive Verhalten

- Desktop ($\geq 1440\text{px}$):** Full-Layout mit allen 5 Zonen
- Tablet (768-1440px):** Pipeline wird vertikal, Analytics overlay-only
- Mobile ($< 768\text{px}$):** Nur Top-Bar + Event-Stream, Pipeline als Mini-Übersicht

9. Tech Stack

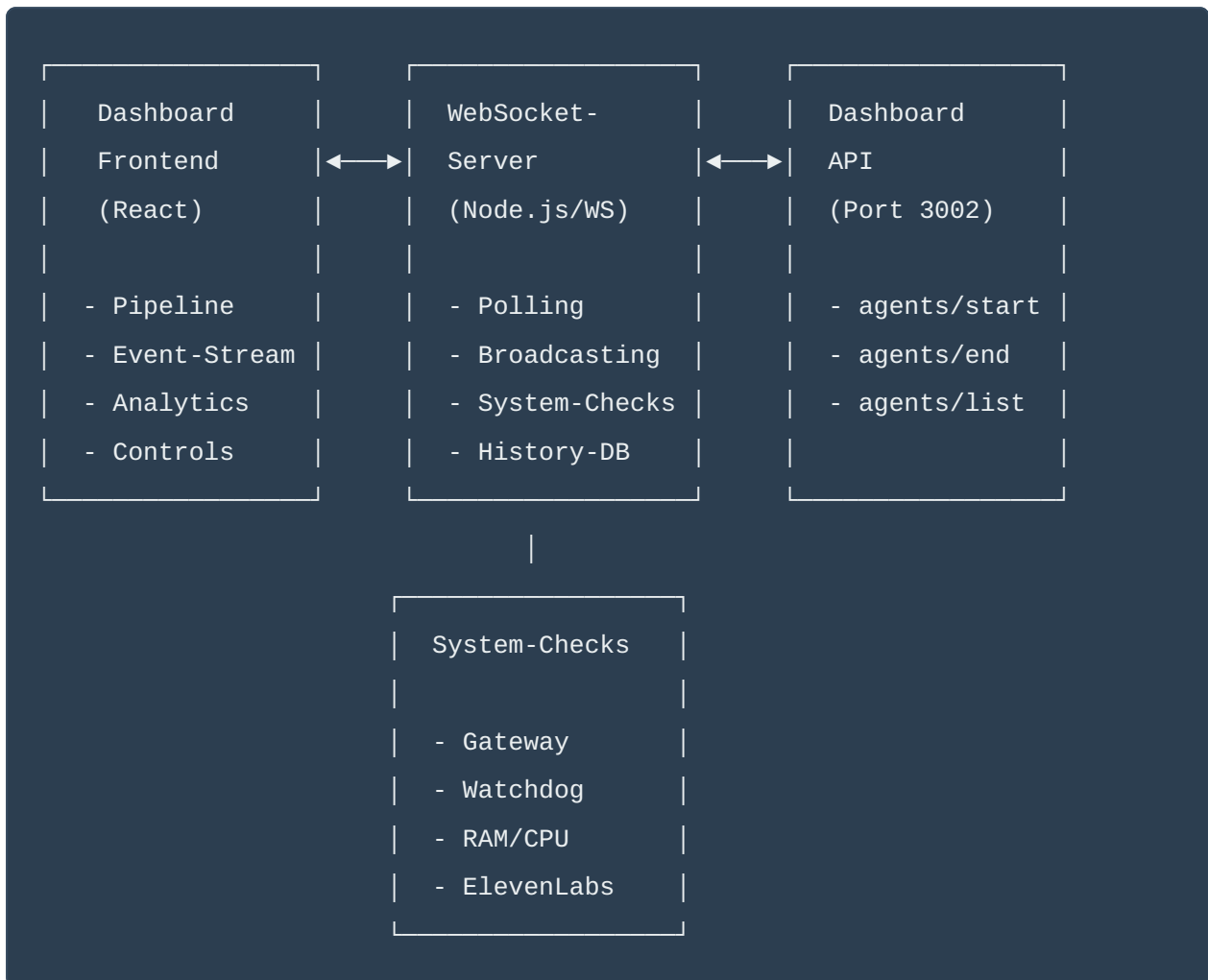
9.1 Frontend

Technologie	Zweck
React 18	UI-Framework
Vite	Build-Tool, Hot-Reload
TailwindCSS	Styling, Design-System
Framer Motion	Animationen, Gesten
Socket.io Client	WebSocket-Verbindung
Recharts	Diagramme & Charts
Lucide Icons	Icon-System

9.2 Backend (WebSocket Server)

Technologie	Zweck
Node.js	Runtime
Express	HTTP-Server
Socket.io	WebSocket-Engine
SQLite	Session-History (lokal)
node-cron	Periodische System-Checks

9.3 Architektur



9.4 Daten-Flow

1. **WebSocket-Server** **pollt** die Dashboard API (`/api/agents/list`) alle 30 Sekunden
 2. **System-Checks** (RAM, CPU, Gateway) laufen alle 30 Sekunden
 3. **Änderungen** werden sofort per WebSocket an alle verbundenen Clients gepusht
 4. **Frontend** empfängt Updates und animiert die UI entsprechend
 5. **History** wird in SQLite gespeichert für Analytics und Gantt-Charts
-

10. Deployment

10.1 Infrastruktur

- **Domain:** `mission.steppa.online` (neue Subdomain)
- **Build-Pfad:** `/var/www/apps/mission-control/dist`
- **WebSocket-Server:** Port 3004 (neuer Port)
- **Caddy:** Reverse-Proxy + WebSocket-Support

10.2 Caddy-Konfiguration

```
handle /mission/* {
    uri strip_prefix /mission
    root * /var/www/apps/mission-control/dist
    try_files {uri} /index.html
    file_server
}

handle /api/ws/* {
    reverse_proxy localhost:3004
}
```

10.3 Deployment-Checkliste

1. Build erfolgreich (`npm run build`)

2. Assets prüfen (JS/CSS Pfade)
 3. Nach `/var/www/apps/mission-control/` kopieren
 4. Caddy Config hinzufügen
 5. WebSocket-Server als systemd-Service einrichten
 6. `systemctl reload caddy`
 7. Browser-Test: <https://mission.steppa.online>
 8. WebSocket-Verbindung testen
-

11. Roadmap

Phase 1: Grundgerüst (Woche 1-2)

- React-App mit Vite + Tailwind aufsetzen
- Top-Bar mit Live-Status
- WebSocket-Server mit Dashboard-API-Polling
- Basic Event-Stream
- Deployment auf mission.steppa.online

Phase 2: Pipeline-Visualisierung (Woche 3-4)

- Pipeline-Layout mit horizontaler Flow
- Agent-Nodes mit Farbkodierung
- Verbindungslinien mit Animation
- Spawn/Complete/Fail-Animationen
- Live Runtime-Timer

Phase 3: Analytics (Woche 5-6)

- SQLite History-Speicherung
- Gantt-Chart für Session-History
- Parallelitäts-Diagramm
- Modell-Performance-Vergleich
- Error-Rate-Tracking

Phase 4: Premium-Features (Woche 7-8)

- AI-Powered Insights (automatische Mustererkennung)
- Telegram-Benachrichtigungen für kritische Events
- Custom Dashboards (User kann Layout anpassen)
- Export (PNG, PDF Reports)
- Dark/Light Theme Toggle
- Mobile-Optimierung

Phase 5: Erweiterte Integrationen (Woche 9+)

- ElevenLabs-Quota-Visualisierung
 - Watchdog-Timeline
 - Caddy-Status-Integration
 - Git-Commit-Tracking (was hat der Agent verändert?)
 - Voice-Status (wenn TTS-Answers generiert werden)
-

12. Technische Risiken & Herausforderungen

12.1 WebSocket-Verbindungsabbrüche

Risiko: Mobile Netzwerke, Server-Restarts, Gateway-Neustarts können WS-Verbindungen unterbrechen.

Mitigation: Auto-Reconnect mit exponentiellem Backoff. State-Sync nach Reconnect (letzten bekannten Stand abholen). Fallback auf HTTP-Polling wenn WS nicht verfügbar.

12.2 Performance bei vielen Agents

Risiko: Bei 50+ gleichzeitigen Agents kann die Pipeline-Ansicht unübersichtlich werden.

Mitigation: Virtualisierung (nur sichtbare Nodes rendern). Gruppierung nach Orchestrator. Zoom & Pan für große Pipelines. Auto-Layout-Algorithmus.

12.3 Daten-Konsistenz

Risiko: Dashboard API und WebSocket-Server können asynchron sein – Events könnten in falscher Reihenfolge ankommen.

Mitigation: Timestamps auf allen Events. Server-seitige Sortierung vor Broadcast. Client-seitige Re-Ordering-Logik.

12.4 Skalierbarkeit

Risiko: Wenn mehrere User gleichzeitig zugreifen, muss der WebSocket-Server stabil bleiben.

Mitigation: Connection-Pooling. Rate-Limiting für System-Checks. Caching von häufig abgerufenen Daten. Load-Testing vor Produktion.

12.5 Datenschutz

Risiko: Agent-Tasks und Prompts können sensible Daten enthalten (API-Keys, persönliche Infos).

Mitigation: Redaction-Filter für sensible Daten. Optionale Verschlüsselung der WebSocket-Verbindung. Keine persistente Speicherung von Prompts in der History-DB ohne Consent.

13. Fazit

Das Mission Control Dashboard ist mehr als ein Monitoring-Tool – es ist die **Brücke zwischen Abstraktion und Anschauung**. Es macht die unsichtbare Arbeit einer KI-Orchestrierung sichtbar, greifbar und verständlich. Für Bastian als Operator bedeutet es: Volle Kontrolle, volle Transparenz, volles Vertrauen. Für das Team bedeutet es: Ein Werkzeug das nicht nur funktioniert, sondern das man gerne benutzt – weil es nicht nur Daten zeigt, sondern Geschichten erzählt.

Jeder Agent-Spawn ist eine Geburt. Jede Completion ist ein Erfolg. Jeder Timeout ist eine Lektion. Das Mission Control Dashboard macht all das sichtbar.

Nächster Schritt: Phase 1 starten – Grundgerüst aufsetzen und WebSocket-Server bauen.

Ende des Konzepts.